

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..1

YOUR SERVERPOD PROJECT

 Copy as Markdown

Your Serverpod project

A Serverpod project is one workspace holding three Dart packages: your server, a generated client, and your Flutter app. You write data models and endpoints on the server; Serverpod generates everything between them, so your app calls the server through typed Dart methods instead of hand-written networking code. Knowing what lives where makes every other page in this section easier to follow.

Running `serverpod create myproject` generates one workspace with three packages:

```
myproject/  
├─ myproject_server/    # Your server: endpoints, models, config, migrations  
├─ myproject_client/    # Generated client: how your app calls the server  
└─ myproject_flutter/   # Your Flutter app
```

That is the whole mental model: you work in the server and the app, and Serverpod maintains the client between them. Everything else in the workspace supports those three, and the sections below introduce each piece as you need it.

► Expand the full workspace tree.

How the packages relate

The Flutter app depends on the client package, and the client is how the app reaches the server: it holds a typed method for every endpoint you write. Neither the server nor the client depends on the other; instead, the server's `config/generator.yaml` points at the client package (`client_package_path`), and code generation writes the client's contents directly.

The workspace is a Dart pub workspace: each package declares `resolution: workspace`, and `dart pub get` at the project root resolves all three.



Ask AI

Where your code lives, and what is generated

You write two kinds of source files on the server, and both can live anywhere under the server's `lib/`:

- **Model files** (`.spy.yaml`) define your serializable classes and database tables. The template keeps them next to the code that uses them, such as `lib/src/greetings/greeting.spy.yaml`. See [Working with models](#).
- **Endpoints** are Dart classes extending `Endpoint`, such as `lib/src/greetings/greeting_endpoint.dart`. Each public method becomes a callable client method: `GreetingEndpoint.hello` is called as `client.greeting.hello(...)`. See [Working with endpoints](#).

The scaffolded project follows the same pattern beyond the greeting example: `lib/src/auth/` holds the endpoints behind the default email sign-in, and `lib/src/web/` holds the routes the web server serves.

From those, `serverpod generate` produces the bridge between server and app:

- On the server, `lib/src/generated/` gets the endpoint dispatch code, the protocol description, the pre-configured `Serverpod` class the server starts from, and a Dart class per model.
- In the client package, `lib/src/protocol/` gets the `Client` class with its typed endpoint methods, plus the same model classes for the app's side.
- In `test/integration/test_tools/`, the generated [test tools](#).

Generated code is overwritten on every run, so never edit it; the scaffolded analyzer config excludes it for that reason. While `serverpod start` runs, generation happens automatically when you save; outside a session, run `serverpod generate` yourself.

The server entry point

The server starts in `bin/main.dart`, which only calls the `run` function in `lib/server.dart`. That function creates the server from the generated `Serverpod` class, registers everything that is not an endpoint, and starts it:

```
void run(List<String> args) async {
  // The generated Serverpod class.
  final pod = Serverpod(args);

  // Registrations go here: authentication services and web routes
  // in the scaffolded project, and anything else you add.

  // Start the server.
  await pod.start();
}
```

Endpoints need no registration: the generated `Serverpod` class already carries them all. Web routes, by contrast, are registered imperatively on `pod.webServer`, which is why the scaffolded `run()` contains route setup. When `pod.start()` runs, the server connects to the database, applies pending migrations when started with `--apply-migrations`, connects to Redis if enabled, brings up its servers, and starts background work such as [future calls](#) and [health checks](#).

The three servers

One Serverpod instance runs up to three servers, each on its own port in development:

- **The API server (8080)** handles incoming endpoint calls from your app. This is the server your generated client talks to.
- **The Insights server (8081)** is a separate, secret-protected server used by Serverpod's tooling. See [Insights](#).
- **The web server (8082)** serves web content: HTML routes, static files, and your built Flutter web app. See [Web server](#).

The scaffolded development config also reserves port 8090 for the database and 8091 for [Redis](#).

The config and migrations directories

The server's `config/` directory holds one YAML file per [run mode](#) (`development`, `test`, `staging`, `production`), the `passwords.yaml` secrets file scaffolded with generated per-environment secrets, and `generator.yaml`, which configures code generation. See [Configuration](#) for all of them.

The `migrations/` directory starts with one migration that creates your initial schema, and grows as your models change. See [Migrations](#).

After the first server start, a gitignored `.serverpod/` directory also appears in the server package: it holds the [embedded database's](#) data files, one subdirectory per run mode, created by the server on demand.

Editor and agent files

The workspace ships ready for IDE debugging and AI agents: `.vscode/` contains attach configurations that connect the debugger to a running `serverpod start` session, and `AGENTS.md` instructs AI agents on how to work in the project.

Each editor you pick at create time gets its own MCP config, which registers Serverpod's [MCP server](#), and a skills folder:

Editor	MCP config	Skills
Claude	<code>.mcp.json</code>	<code>.claude/skills/</code>
Cursor	<code>.cursor/mcp.json</code>	<code>.cursor/skills/</code>
VS Code	<code>.vscode/mcp.json</code>	<code>.agents/skills/</code>
Codex	<code>.codex/config.toml</code>	<code>.agents/skills/</code>
OpenCode	<code>opencode.json</code>	<code>.opencode/skills/</code>
Antigravity	<code>.agents/plugins/serverpod-local/</code>	<code>.agents/skills/</code>

The setup screen preselects no editors, so pick the ones you use. With `--no-interactive`, `serverpod create` configures Claude, Cursor, and VS Code unless you pass `--ide`.

The workspace `.gitignore` excludes these files for every editor except OpenCode, so Git picks up `opencode.json` and `.opencode/` unless you add them to `.gitignore`.

Related

- [How it works](#): the guided tour of building with Serverpod.
- [Running your server](#): the development loop around this project.
- [Configuration](#): every file in `config/` in depth.
- [Working with endpoints](#): writing the API the client calls.
- [Working with models](#): defining your data.